CSS: Cascaded Style Sheet

Ambient intelligence course (A.Y. 2025/2026)
Topic taught by Prof. Floriano Scioscia

Gabriele Colapinto

April 16, 2026

License

Notes from Ambient intelligence (A.Y. 2025/2026) by Gabriele Colapinto. Course taught by professors Michele Ruta, Floriano Scioscia, Arnaldo Tomasino, Davide Loconte — Licensed under CC BY-NC 4.0.
https://creativecommons.org/licenses/by-nc/4.0/

© 2025 Gabriele Colapinto
Released under the Creative Commons Attribution–NonCommercial 4.0 International License

Contents

1 Applying the Style: CSS Fundamentals **5**
1.1 How CSS Works: The Cascade . . . 5
1.2 The Language of CSS: Rules, Selectors, and Properties . . . 6
1.3 Common Properties and Values . . . 7
1.3.1 Size Units . . . 8
1.3.2 Color Values . . . 8
1.3.3 Font family . . . 8

2 The CSS Box Model: The Foundation of Layout **9**
2.1 Customizing the Border . . . 9

3 Controlling Element Position and Stacking Order **13**
3.1 The Property . . . 13
3.1.1 static . . . 13
3.1.2 relative . . . 13
3.1.3 fixed . . . 13
3.1.4 absolute . . . 14
3.2 Managing overlapping elements . . . 14

4 An Introduction to Responsive Web Design (RWD) **15**

5 Media Queries: Adapting to the Device **17**
5.1 Syntax and Implementation . . . 17
5.1.1 Primary Media Types . . . 18
5.1.2 Common Media Features . . . 18
5.2 Understanding Device Pixel Ratio (DPR) . . . 18

6 Modern CSS Layout Techniques **21**
6.1 The Grid View System . . . 21
6.2 Flexbox: The Flexible Box Layout Model . . . 22
6.2.1 Properties for the Flex Container . . . 23
6.2.2 Properties for Flex Items . . . 25

7 Dynamic Visuals with CSS Animations **27**

8 A Note on Rendering Engines and Prefixes **29**

1 Applying the Style: CSS Fundamentals

CSS transforms a basic HTML structure into a polished and visually engaging user experience. It dictates everything from typography and color to layout and spacing. Understanding the core mechanics of CSS—its syntax, application methods, and the powerful selection system that connects styles to specific HTML elements—is essential for creating modern, well-designed websites.

1.1 How CSS Works: The Cascade

The "Cascading" in Cascading Style Sheets is a crucial concept. It refers to the ordered process by which styles are applied to a document. Rules that are read later in this process can override rules defined earlier, creating a waterfall-like flow of style inheritance and precedence.

There are three primary methods for inserting a style sheet into an HTML document:

1. **External Style Sheet:** Styles are defined in a separate `.css` file and linked to the HTML document using the `<link>` tag within the `<head>` section. This is the industry best practice, as it cleanly separates content from presentation and allows a single file to style an entire website.

```
<head>
    <link rel="stylesheet" type="text/css" href="mystyle.css">
</head>
```

Listing 1.1: Linking of an external style sheet

2. **Internal Style Sheet:** Styles are placed directly within the `<head>` section of an HTML document, inside a `<style>` tag. This method is useful for single-page styling but is less scalable than external sheets.

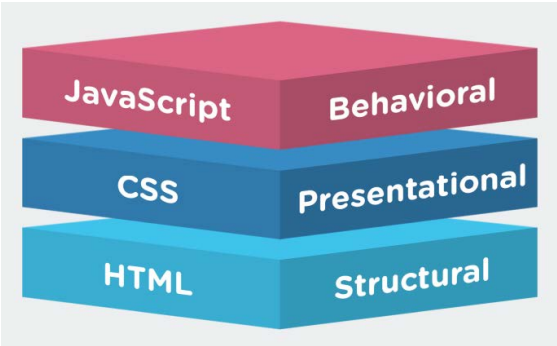

Figure 1.1: Web page stack

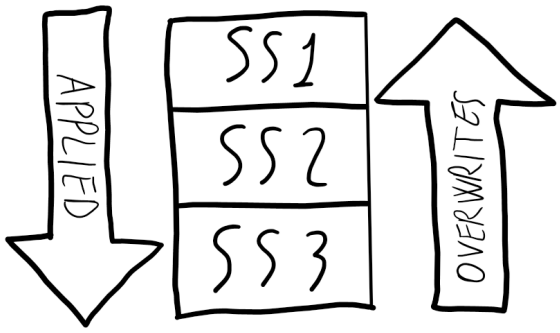

Figure 1.2: CSS cascading order

```
<head>
    <style>
        h1 {
            color: blue;
        }
    </style>
</head>
```

Listing 1.2: Style sheet definition in the head tag

3. **Inline Style:** Styles are applied directly to a single HTML element using the `style` attribute. This approach mixes content with presentation and should be used sparingly, as it has the highest priority and can be difficult to override.

```
<h1 style="color: blue;">This is a heading.</h1>
```

Listing 1.3: Inline element style definition

The priority of these styles follows a specific cascading order, from lowest to highest (See figure 1.2):

1. **Browser default:** Every browser has its own default styles for HTML elements.
2. **External and internal style sheets:** Styles defined in the document's `<head>`. If there are conflicting rules, the one that appears last takes precedence.
3. **Inline style:** Styles applied directly to an element with the `style` attribute have the highest priority and will override rules from all other sources.

1.2 The Language of CSS: Rules, Selectors, and Properties

A CSS style sheet consists of a set of rules. Each rule has two main parts: a **selector** and a **declaration block** (See figure 1.3). The selector points to the HTML element(s) you want to style, while the declaration block, enclosed in curly braces {}, contains one or more property-value pairs separated by semicolons.

CSS provides a rich set of selectors for targeting elements with precision:

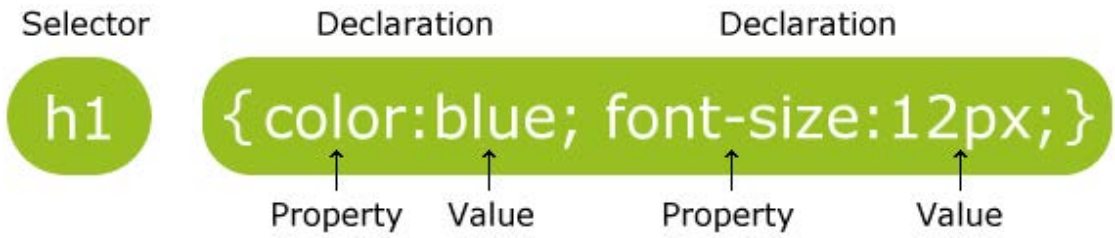

Figure 1.3: CSS syntax

- **Basic Selectors**
  - `element`: Selects all elements of a specific type (e.g., `p` selects all paragraphs).
  - `id` (`#id`): Selects the single element with a specific, unique `id` attribute.
  - `class` (`.class`): Selects all elements that have a specific `class` attribute.
- **Attribute Selector**
  - `[attribute]`: Selects all elements that possess a given attribute (e.g., `p[name]` selects paragraphs with a `name` attribute).
- **Proximity & Combinator Selectors**
  - **Descendant** (`div h1`): Selects all `<h1>` elements that are located anywhere inside a `<div>` element.
  - **Direct Child** (`div > h1`): Selects only `<h1>` elements that are direct children of a `<div>`.
  - **Adjacent Sibling** (`h1 + h2`): Selects an `<h2>` element that is placed immediately after an `<h1>` element at the same hierarchical level.
- **Pseudo-class Selector**
  - `a:hover`: Selects an element based on its current state, such as when a user hovers their mouse over a link.
- **Pseudo-element Selector**
  - `p::first-letter`: Selects and styles a specific part of an element, such as the first letter of a paragraph.
- **Grouping & Universal Selectors**
  - **Grouping** (`h1, h2`): Applies the same style rules to multiple selectors by separating them with a comma.
  - **Universal** (`*`): Selects every single element on the page.

1.3 Common Properties and Values

CSS properties accept a wide variety of values, including units for size and multiple notations for color.

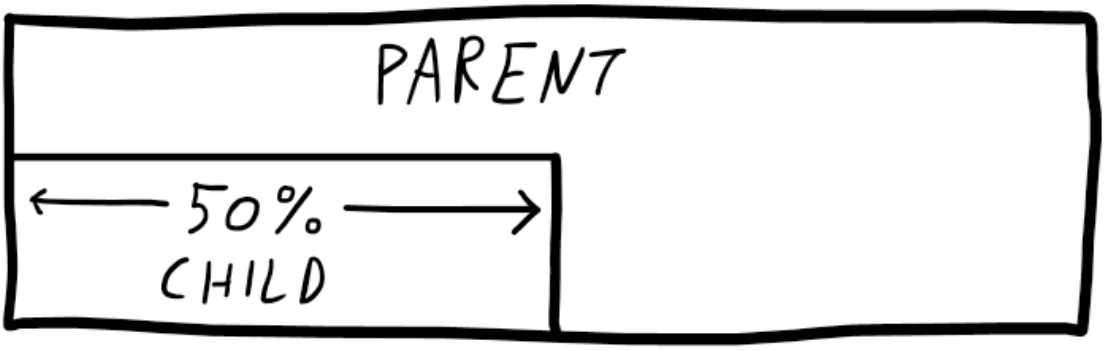

Figure 1.4: An example of an element with a width equal to the 50% of the width of the parent container element.

1.3.1 Size Units

- **Absolute Units:** These are fixed and do not change based on other elements.
  - `px`: Logical pixels, the most common unit for screen-based design.
  - `pt`: Typographic points (1pt = 1/72 of an inch), typically used for print media.
  - `in`: Inches.
  - `cm`
  - `mm`
- **Relative Units:** These are flexible and scale in relation to another value.
  - `em`: Relative to the font-size of the element itself.
  - `%`: Relative to the size of the parent container element. (See figure 1.4)

1.3.2 Color Values

Color can be specified in several ways:

- **Predefined Name:** Using a standard color name (e.g., `red`, `blue`).
- **RGB Function:** Using the `rgb()` function to specify values (0-255) for Red, Green, and Blue channels (e.g., `rgb(255, 0, 0)`).
- **Hexadecimal Notation:** A six-digit code preceded by a hash (`#`) that represents the RGB values in hexadecimal format (e.g., `#FF0000`). Each pair of characters corresponds to the Red, Green, and Blue values, where `FF` is equivalent to 255.

1.3.3 Font family

The `font-family` property is used to specify the typeface for text. It accepts a prioritized list of font names, known as a "font stack." The browser will attempt to use the first font in the list; if it is not available on the user's system, it will try the next one, and so on. It is a crucial best practice to end the list with a generic font family (e.g., `sans-serif` or `serif`) as a fallback to ensure the text is always rendered in a suitable style.

2 The CSS Box Model: The Foundation of Layout

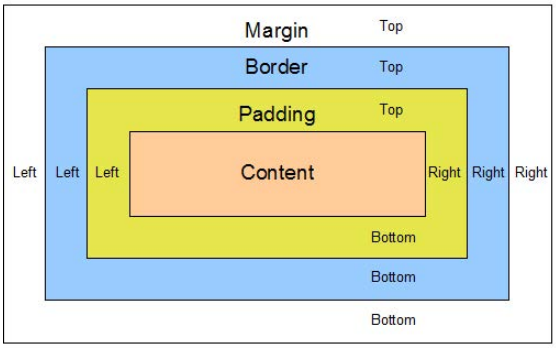

Figure 2.1: CSS box model

The CSS Box Model (figure 2.1) is the fundamental concept governing how every visible HTML element is rendered as a rectangular area on a web page. When a browser displays a document, it assigns a box to each element, and understanding the components of this box is critical for controlling element dimensions, spacing, and overall page layout. Mastering the box model is the first step toward building predictable and well-structured designs.

The model is composed of four primary components, layered from the inside out:

- **Content:** The innermost layer where the actual content of the element, such as text and images, appears.
- **Padding:** The transparent area surrounding the content, creating a buffer *inside* the element's border.
- **Border:** A line that goes around the padding and content. Its appearance can be customized in terms of style, color, and thickness.
- **Margin:** The transparent area outside the border, creating space *between* this element and any adjacent elements.

2.1 Customizing the Border

The appearance of an element's border is specified by the `border-style` property (See figure 2.2 and table 2.1). This property accepts a range of keyword values that define the visual style of the line drawn around the element's padding and content.

Once we understand how every element is represented as a box, we can explore how CSS allows us to precisely position these boxes on the page.

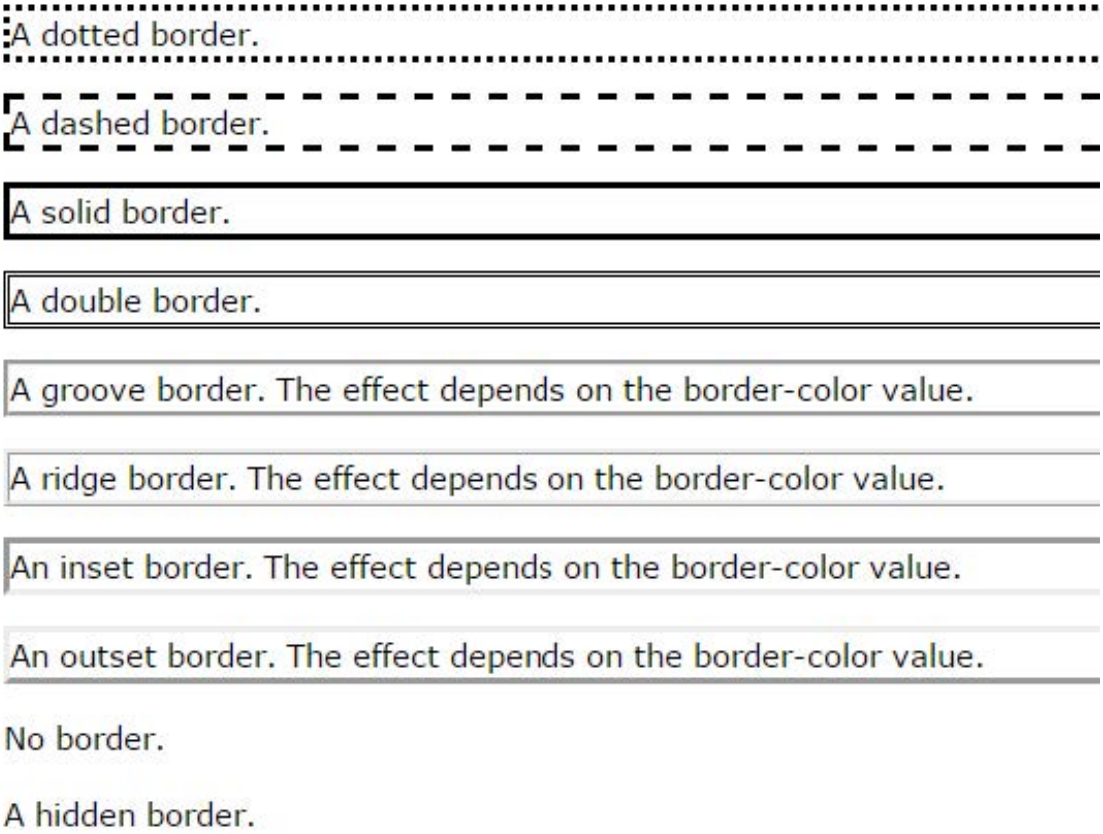

Figure 2.2: Border styles

Style Value	Description
`dotted`	Defines a border composed of dots.
`dashed`	Defines a border composed of dashes.
`solid`	Defines a solid, single-line border.
`double`	Defines a double border.
`groove`	Defines a 3D grooved border. The effect depends on the border-color value.
`ridge`	Defines a 3D ridged border. The effect depends on the border-color value.
`inset`	Defines a 3D inset border, creating the appearance of being embedded.
`outset`	Defines a 3D outset border, creating the appearance of coming out.
`none`	Defines no border.
`hidden`	Defines a hidden border.

Table 2.1: Border Style Values

3 Controlling Element Position and Stacking Order

Beyond defining the space an element occupies, CSS provides powerful properties to control its exact placement on the page. You can position an element within the normal document flow, fix it relative to the browser's viewport, or place it relative to its parent or other ancestor elements. This level of control is essential for creating complex and interactive layouts.

3.1 The Property

The `position` property specifies the positioning method used for an element. It can take one of four primary values.

3.1.1 static

This is the default value. Elements are positioned according to the normal flow of the page. By default, block elements are stacked vertically, one after the other, while inline elements are aligned horizontally in a row.

```
div.static {
    position: static;
}
```

3.1.2 relative

The element is positioned relative to its normal, static position in the document flow. You can then use the offset properties `left`, `top`, `bottom`, and `right` to move it away from that original spot without affecting the layout of surrounding elements.

```
div.relative {
    position: relative;
    left: 30px;
}
```

3.1.3 fixed

The element is positioned relative to the viewport (the browser window). It remains in the same place even when the page is scrolled. This is often used for persistent user interface elements like navigation bars, cookie policy banners, or chatbot buttons that should always be accessible.

```
div.fixed {
    position: fixed;
    bottom: 0;
```

```
    right: 0;
    width: 300px;
}
```

3.1.4 absolute

The element is positioned relative to its nearest positioned ancestor (i.e., the closest parent element that has a `position` value other than `static`). If no such ancestor exists, the element is positioned relative to the initial containing block, which is the **viewport**.

```
div.absolute {
    position: absolute;
    top: 80px;
    right: 0;
    width: 200px;
    height: 100px;
}
```

3.2 Managing overlapping elements

When you manually control element positions, it's possible for their boxes to overlap. The `z-index` property specifies the stack order of these overlapping elements, determining which one appears on top. An element's `z-index` can be a positive or negative integer, with a default value of `0`.

The stacking order is determined by two simple rules:

1. An element with a greater `z-index` value is always rendered in front of an element with a lower value.
2. If two overlapping elements have the same `z-index`, the one that appears last in the HTML code is rendered on top of the other.

While manual positioning provides fine-grained control, modern web development often requires layouts that adapt automatically to different devices and screen sizes, which leads to the principles of Responsive Web Design.

4 An Introduction to Responsive Web Design (RWD)

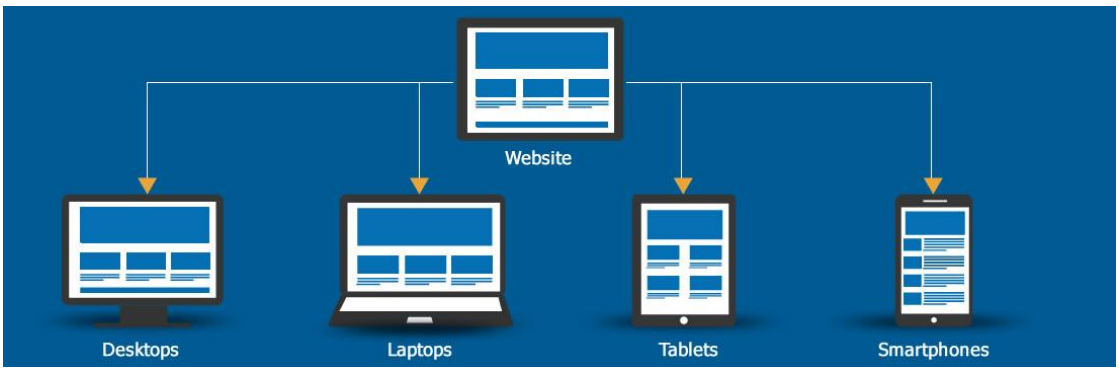

Figure 4.1: Responsive Web Design

Responsive Web Design (RWD) is a design approach that uses only HTML and CSS to create web pages that render well on a variety of devices and screen sizes (See figure 4.1). A key benefit of RWD is that it enables a single codebase to support desktops, tablets, and mobile phones, with the layout dynamically adapting to the user's viewing environment. This eliminates the need for separate websites for different devices.

A critical first step in implementing RWD is to include the viewport `<meta>` tag in the `<head>` of your HTML document. This tag instructs the browser on how to control the page's dimensions and scaling (See figure 4.2).

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- `width=device-width` sets the width of the page's viewport to match the screen-width of the device.
- `initial-scale=1.0` sets the initial zoom level when the page is first loaded by the browser.

With the viewport properly configured, developers can leverage modern CSS3 techniques to build responsive layouts. The two primary methods for achieving this are **Media Queries**, which apply different styles based on device characteristics, and the **Flexible Box Layout (Flexbox)**, a layout model for aligning and distributing space among items in a container.

The next section will explore Media Queries in detail, as they are the core mechanism for adapting a design to the specific capabilities of a device.

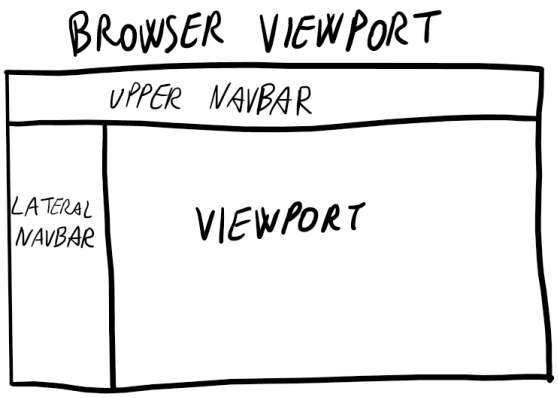

Figure 4.2: Browser viewport

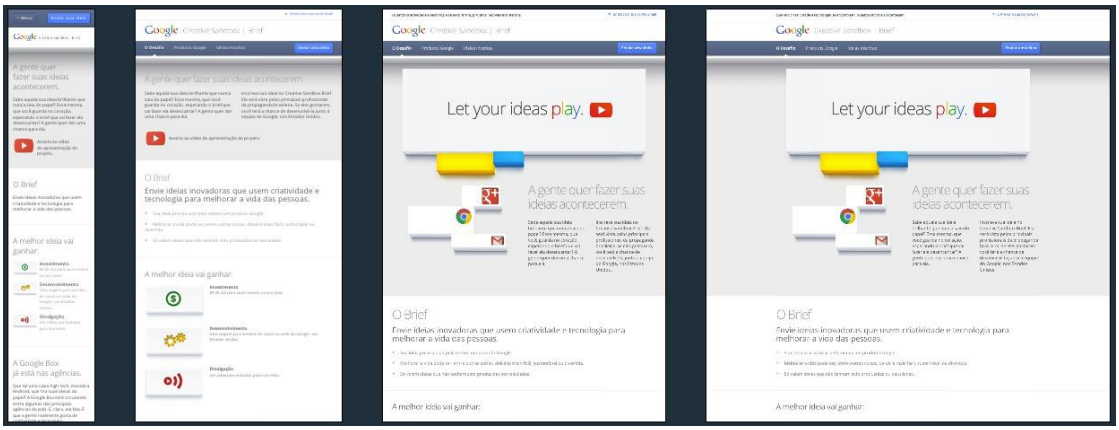

Figure 4.3: Example of Responsive Web Design

5 Media Queries: Adapting to the Device

Media Queries are a CSS3 feature that significantly extends the concept of media types found in CSS2. Instead of just targeting a broad device category like `print` or `screen`, Media Queries allow styles to be applied based on the specific capabilities and characteristics of a device. This enables developers to create highly tailored experiences by testing features like viewport width, device resolution, and screen orientation.

5.1 Syntax and Implementation

A media query consists of a media type and one or more expressions that check for specific media features. If the conditions are met, the CSS rules within the query are applied.

The general syntax for the `@media` at-rule is:

```
@media not|only mediatype and (expressions) { ... }
```

Listing 5.1: @media at-rule general syntax

Examples of `@media` at-rule are:

```
@media all and (min-width: 1024px) and (max-width:1280px) {/*** CSS-Code; ***/}

/* Portrait*/
@media only screen
and (min-device-width: 414px)
and (max-device-width: 736px)
and (-webkit-min-device-pixel-ratio: 3)
and (orientation: portrait) {
    /*** CSS-Code; ***/
}

/* Landscape*/
@media only screen
and (min-device-width: 414px)
and (max-device-width: 736px)
and (-webkit-min-device-pixel-ratio: 3)
and (orientation: landscape) {
    /*** CSS-Code; ***/
}
```

Listing 5.2: @media at-rule examples

There are two primary ways to implement media queries:

1. **Within a single stylesheet:** You can include multiple `@media` rules directly in your main CSS file. This is the most common approach.

2. **Linking separate stylesheets:** You can create different CSS files for different media types and link them in your HTML using the `<link>` tag's `media` attribute.

5.1.1 Primary Media Types

Media Type	Description
`all`	Used for all media type devices.
`print`	Used for printers, allowing for printer-friendly page layouts.
`screen`	Used for computer screens, tablets, and smart-phones.
`speech`	Used for screen readers that read the page content aloud.

Table 5.1: Primary Media Types

5.1.2 Common Media Features

Media Feature	Description
`width`	The width of the viewport.
`height`	The height of the viewport.
`device-width`	The width of the device's screen.
`device-height`	The height of the device's screen.
`orientation`	The orientation of the viewport (either `portrait` or `landscape`).
`resolution`	The resolution of the output device (in `dpi` or `dppx`).

Table 5.2: Common Media Features

5.2 Understanding Device Pixel Ratio (DPR)

Device Pixel Ratio (DPR) is the ratio between the physical pixels on a hardware screen and the logical (CSS) pixels used for layout. This ratio is crucial for ensuring that images and graphics appear sharp on high-resolution displays, such as Apple's Retina screens.

The formula for DPR is: `DPR = Physical Pixels / CSS Pixels`

DPR is important because ignoring it can lead to blurry visuals. For example, if a 100px by 100px image is displayed on a device with a DPR of 2.0, each logical pixel is stretched to fit a 2x2 grid of physical pixels. This stretching causes the image to appear pixelated and less crisp. Imagine trying to create a sharp photograph using only four large Lego bricks; the result is blocky and indistinct. This is analogous to how a low-resolution image appears on a high-DPR screen.

To target devices with different DPRs, you can use media queries to serve higher-resolution assets. There are two primary CSS methods for this:

1. **The standard `resolution` feature:** This is the modern, standardized approach. The `dppx` unit stands for "dots per px".
2. **The non-standard `-webkit-min-device-pixel-ratio` feature:** This prefixed version was introduced by the WebKit rendering engine and is still necessary for ensuring compatibility with some older WebKit-based browsers.

After querying a device's properties to determine which styles to apply, the next step is to use robust layout systems that can effectively organize content based on that information.

6 Modern CSS Layout Techniques

Early web layouts often relied on HTML `<table>` elements to structure page content. This technique is now considered deprecated because it is semantically incorrect and highly inflexible, making it unsuitable for responsive design. Modern CSS provides dedicated layout modules that offer powerful solutions for creating robust and adaptive designs. The two most prominent techniques are CSS Grid Layout and the Flexible Box Layout (Flexbox). It's critical to understand their distinct purposes: **Flexbox** is a one-dimensional system, ideal for laying out items in a single row or column, such as navigation bars or form elements. **Grid** is a two-dimensional system, designed to control layout in both rows and columns simultaneously, making it perfect for overall page structure.

6.1 The Grid View System

A common conceptual approach to layout, popularized by frameworks like Bootstrap, involves dividing the page into a set number of columns, typically 12. This system helps developers place elements on the page by assigning them CSS classes that correspond to a specific number of columns the element should occupy (See figure 6.1).

Its primary purpose is to simplify the creation of organized, column-based layouts. Elements are arranged in rows, and their widths are defined by the number of grid columns they span. This system has a built-in responsive behavior: if the sum of the column classes in a single row exceeds the total available (e.g., 12), the overflowing element automatically reflows onto the next line. This allows the layout to adapt fluidly to different viewport sizes. It is important to differentiate this framework-based approach from the native **CSS Grid Layout** module, which is a more powerful, built-in browser feature for two-dimensional layouts that does not rely on predefined classes.

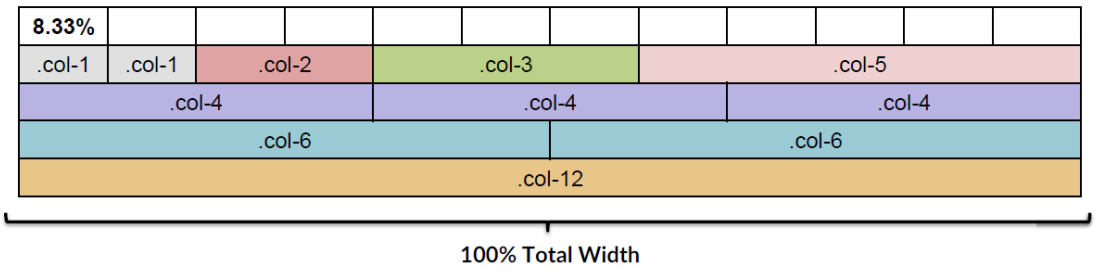

Figure 6.1: Grid view system

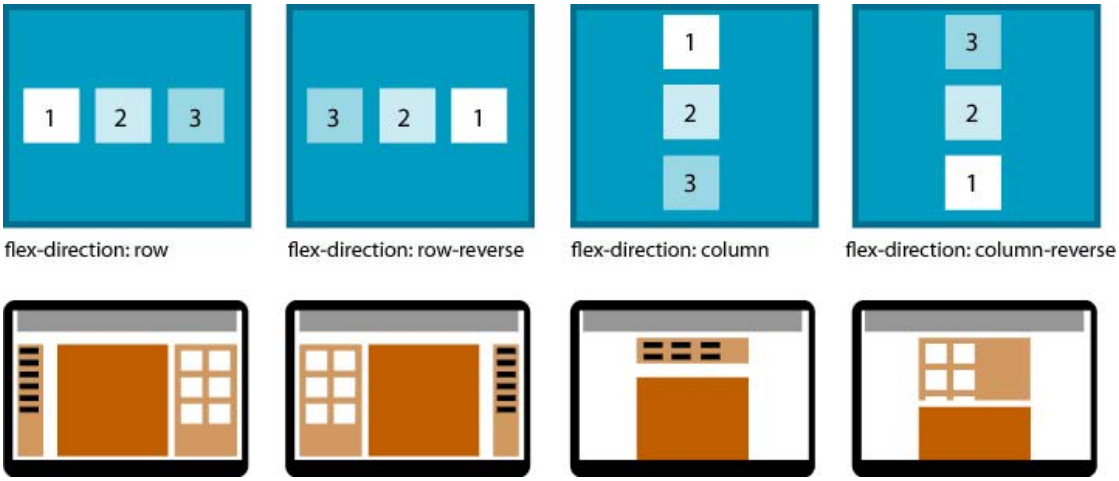

Figure 6.2: Flexible box layout

6.2 Flexbox: The Flexible Box Layout Model

Flexbox, or the Flexible Box Layout model, is a one-dimensional layout model designed to provide a more efficient way to distribute space and align items within a container. It excels at managing the layout of components and small-scale structures within a larger design.

The Flexbox model is built on two core components:

- **Flex Container:** The parent element that holds the flex items. A flex container is created by setting its `display` property to `flex` or `inline-flex`.
- **Flex Items:** The direct children of the flex container. These items are laid out and aligned according to the rules set on the container.

Flexbox operates along two primary axes, which define the direction of the layout:

- **Main Axis:** The primary axis along which flex items are laid out. The `flex-direction` property defines this axis, which can be horizontal (`row`) or vertical (`column`).
- **Cross Axis:** The axis that runs perpendicular to the main axis.

6.2.1 Properties for the Flex Container

The following properties are applied to the parent element (the flex container) to control the layout of its children.

Property	Description	Common Values
`display`	Defines the element as a flex container.	`flex`, `inline-flex`
`flex-direction`	Establishes the main axis, defining the direction items are placed.	`row`, `row-reverse`, `column`, `column-reverse`
`flex-wrap`	Controls whether flex items wrap onto a new line if they overflow. (See figures 6.3 and 6.4)	`nowrap`, `wrap`, `wrap-reverse`. (`nowrap` forces all the child elements to be in the same line. In case the child elements overflow the container they might become inaccessible to the user so, in case we decide to use the `nowrap` option, it is advisable to use the `overflow: scroll` option referring to the main axis of the flexbox.)
`flex-flow`	A shorthand for `flex-direction` and `flex-wrap`.	`row wrap`, `column nowrap`
`justify-content`	Aligns flex items along the main axis, distributing extra space.	`flex-start`, `flex-end`, `center`, `space-between`, `space-around`, `space-evenly`
`align-items`	Aligns flex items along the cross axis.	`stretch`, `flex-start`, `flex-end`, `center`, `baseline`
`align-content`	Aligns a container's lines when there is extra space on the cross axis, similar to how `justify-content` aligns items on the main axis. Only affects multi-line containers (`flex-wrap: wrap`).	`flex-start`, `flex-end`, `center`, `space-between`, `space-around`, `stretch`

Table 6.1: Properties for the Flex Container

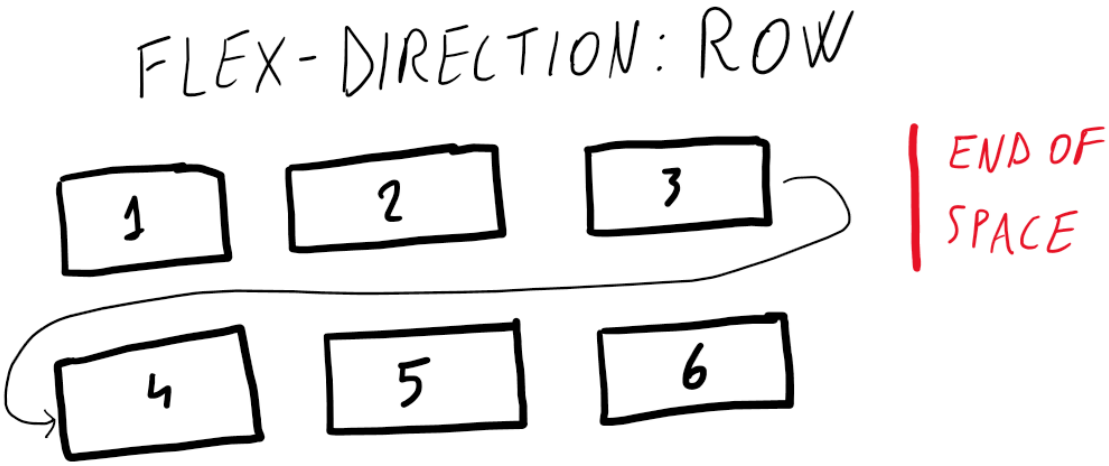

Figure 6.3: Flexbox row overflow with wrap

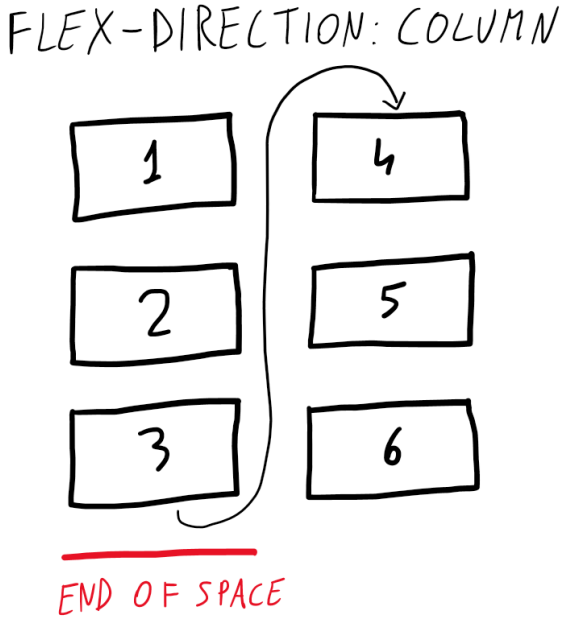

Figure 6.4: Flexbox column overflow with wrap

6.2.2 Properties for Flex Items

These properties are applied to the child elements (the flex items) to control their individual behavior within the container.

Property	Description	Common Values
`order`	Controls the order in which items appear in the container.	`<integer>` (e.g., -1, 0, 1)
`flex-grow`	Defines the ability of an item to grow, taking up available space.	`<number>` (e.g., 0, 1, 2)
`flex-shrink`	Defines the ability of an item to shrink if there is not enough space.	`<number>` (e.g., 0, 1, 2)
`flex-basis`	Defines the default size of an item before remaining space is distributed.	`<length>` (e.g., 200px, 50%, auto)
`flex`	A shorthand for `flex-grow`, `flex-shrink`, and `flex-basis`.	`0 1 auto` (default), `1`, `none`
`align-self`	Overrides the default alignment set by `align-items` for an individual item.	`auto`, `flex-start`, `flex-end`, `center`, `stretch`, `baseline`

Table 6.2: Properties for Flex Items

Beyond static layouts, CSS also provides powerful tools for creating dynamic visual effects, such as animations.

7 Dynamic Visuals with CSS Animations

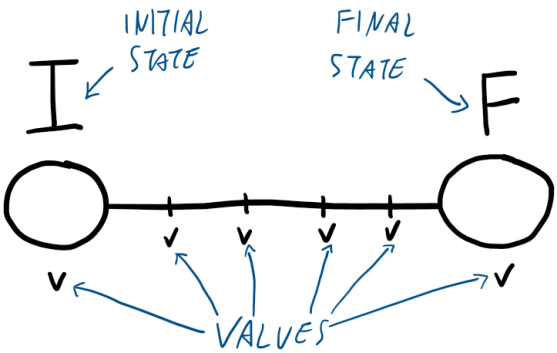

Figure 7.1: CSS animation divided in steps

CSS Animations are a feature that allows the transformation of an element's properties from an initial state to a final state over a specified duration. This makes it possible to create complex, multi-step visual effects like movement, color changes, and fading without relying on JavaScript.

The core of a CSS animation is the `@keyframes` at-rule. This rule is used to define the animation's intermediate steps, or "keyframes", as percentages of the overall duration (See figure 7.1). For simple transitions between a start and end state, the keywords `from` (equivalent to 0%) and `to` (equivalent to 100%) can be used instead of percentages.

A conceptual `@keyframes` rule looks like this:

```
@keyframes myAnimation {
  from { background-color: red; }
  to   { background-color: yellow; }
}
```

Once an animation is defined with `@keyframes`, it can be applied to an element using several key properties:

- `animation-name`: Specifies the name of the `@keyframes` rule to apply.
- `animation-duration`: Sets how long the animation should take to complete one cycle.
- `animation-delay`: Specifies a delay before the animation starts.

Understanding how CSS rules are defined is one part of the equation; the other is understanding the software that interprets and renders them.

8 A Note on Rendering Engines and Prefixes

A browser's **rendering engine** is the core software component responsible for interpreting HTML, CSS, and other web content to calculate the layout and render the visual representation of a web page. Different browsers may use different rendering engines.

WebKit is a prominent open-source rendering engine initially created by Apple. It powers browsers like Apple's Safari and was formerly used by Google Chrome. Rendering engines are where new or experimental CSS features are often first implemented.

To implement these non-standard or experimental features without conflicting with existing standards, rendering engines use **vendor-specific prefixes**. A common example is the `-webkit-` prefix, used by the WebKit engine. For instance, the `-webkit-device-pixel-ratio` media feature was a non-standard feature introduced by WebKit to handle high-DPR displays before a standardized equivalent was widely adopted.

While these prefixes are sometimes necessary to ensure broad compatibility with older browsers, developers should prioritize using standard, unprefixed CSS features (like the `resolution` media feature) whenever possible. Standard features are supported across all modern rendering engines, leading to more maintainable and future-proof code.